Lec_07_threads.pptx
Presentation

"Act as an expert educator. Create [Number] challenging true/false questions on [Given Uploaded Slides].
Rules:
Avoid negative statements or double negatives.
Ensure the 'False' statements are plausible, not obviously wrong.
Provide a detailed explanation in simple english for why the answer is True or False.
Focus on key concepts, not trivial details."

Here are **15 challenging True/False questions** based on your uploaded slides on **Threads** Lec_07_threads .

(Each includes a simple-English explanation and focuses on key concepts.)

---

1) Statement

A thread is the basic unit that the CPU scheduler chooses to run.

Answer: True

Explanation: The slides describe a thread as the basic unit of CPU utilization and the entity scheduled on the CPU. A process may contain multiple threads, but the scheduler runs threads.

---

2) Statement

In a multithreaded process, each thread has its own program counter and its own stack.

Answer: True

Explanation: Each thread needs its own “where am I executing?” (program counter) and its own function-call history/local variables (stack). That’s why these are per-thread.

---

3) Statement

All threads in the same process share the same heap and data section.

Answer: True

Explanation: Threads of one process work in the same address space, so shared regions like heap and data are common to all threads in that process.

---

4) Statement

Threads provide resource sharing because they can naturally access the same code and data without using IPC.

Answer: True

Explanation: Since threads share the process address space, they can share data directly. Separate processes usually need IPC (pipes, shared memory, sockets) to exchange data.

---

5) Statement

Creating and switching threads is usually cheaper than creating and switching processes.

Answer: True

Explanation: The slides highlight “economy of effort”: thread creation and switching typically have lower overhead than process creation and switching.

---

6) Statement

A key disadvantage of threads is that a serious crash in one thread can terminate the entire process.

Answer: True

Explanation: Threads live inside one process. If one thread causes a fatal error (like a segmentation fault), the OS terminates the whole process, which ends all threads.

---

7) Statement

Because threads share memory, race conditions can happen when multiple threads update shared data.

Answer: True

Explanation: Shared memory is powerful but risky. If two threads write the same shared variable without coordination, results can become unpredictable.

---

8) Statement

Thread-safe functions are designed so multiple threads can use them without causing data inconsistency.

Answer: True

Explanation: The slides warn that many library functions are not thread-safe because they use shared global/static data. Thread-safe functions avoid that, or use synchronization internally.

---

9) Statement

In user-level threading, the kernel schedules each user thread directly.

Answer: False

Explanation: In user-level threading, the kernel usually sees only the process (or one kernel thread). The user-level thread library schedules user threads, not the kernel.

---

10) Statement

In user-level threading, if one user thread blocks on a blocking system call, other user threads in that process may also stop running.

Answer: True

Explanation: The kernel doesn't know there are multiple user threads. If the process blocks (because one user thread made a blocking call), the whole process blocks.

---

11) Statement

Kernel-level threads allow other threads in the same process to continue even if one thread blocks on I/O.

Answer: True

Explanation: With kernel threads, the OS knows about each thread. If one kernel thread blocks, the kernel can schedule another runnable thread from the same process.

---

12) Statement

In the Many-to-One (M:1) model, many user threads map to a single kernel thread, so true parallelism on multicore CPUs is not achieved.

Answer: True

Explanation: The kernel schedules only one kernel thread, so only one can run at a time on the CPU cores, even if the program has many user threads.

---

13) Statement

In the One-to-One (1:1) model, each user thread maps to a kernel thread, which supports parallel execution but increases overhead.

Answer: True

Explanation: More kernel threads means better parallelism and blocking behavior, but also more kernel involvement and resource usage.

---

14) Statement

The Many-to-Many (M:N) model tries to balance the low overhead of M:1 with the better concurrency of 1:1, but it is more complex to implement.

Answer: True

Explanation: That’s exactly the trade-off described: it avoids major weaknesses of M:1 and 1:1, but adds complexity and is less common.

---

15) Statement

If a thread in a multithreaded program calls `exec()`, only that thread is replaced, and the other threads keep running.

Answer: False

Explanation: `exec()` replaces the entire process image (memory + program). That means all existing threads are terminated because the whole process is overwritten by the new program.

If you tell me **[Number]** you want (e.g., 20, 30, 50), I can generate that many in the same style from the same slides Lec_07_threads .

Sources

20

Here are **20 challenging True/False questions** based on your uploaded lecture slides on **Threads** Lec_07_threads . Each includes a clear explanation in simple English.

---

1) Statement

Flow of execution is the path a program follows from start to finish, including branches and function calls.

Answer: True

Explanation: The slides define flow of execution (flow of control) as the sequence/path of executed instructions, including branches and calls.

---

2) Statement

Execution context is the state of a program at a specific moment, including CPU registers and memory contents.

Answer: True

Explanation: The slides describe execution context as “state at a point in time,” including registers (PC, SP, etc.) and memory (stack/heap variables), plus open files and I/O state.

---

3) Statement

Every process has a unique flow of execution and a unique execution context.

Answer: True

Explanation: This is stated directly: a process has its own flow of execution and context.

---

4) Statement

Parallel execution requires multiple CPUs/cores, while concurrent execution can happen on a single CPU by rapid switching.

Answer: True

Explanation: Parallel means truly running at the same time on multiple processors; concurrent means time-shared switching on one CPU.

---

5) Statement

A thread is a single flow of execution used to perform a particular task within a process.

Answer: True

Explanation: The slides define a thread this way and explain that a multithreaded process can do multiple tasks at once.

---

6) Statement

A thread is the basic unit of CPU utilization and is what the CPU scheduler schedules to run.

Answer: True

Explanation: The slides explicitly say the thread is the scheduled entity.

---

7) Statement

Each thread has its own program counter, stack, and register set.

Answer: True

Explanation: Threads need their own “current instruction” (PC), call history (stack), and CPU state (registers).

---

8) Statement

Threads in the same process share the text section, data section, heap, and open files.

Answer: True

Explanation: The slides list these as shared resources among threads of the same process.

---

9) Statement

Multithreading always means threads run in parallel at the same time.

Answer: False

Explanation: On a single CPU, threads usually run **concurrently** via rapid switching. Parallel execution requires multiple CPUs/cores.

---

10) Statement

Creating multiple processes is often a good choice when tasks must be isolated, but it usually needs IPC to share data.

Answer: True

Explanation: The slides explain processes are good for isolation but require IPC to share, and are heavier to create/manage.

---

11) Statement

Threads are often called “lightweight processes” because creating threads is usually lower cost than creating processes.

Answer: True

Explanation: The slides emphasize lower time/memory cost and mention examples like much faster thread creation than process creation.

---

12) Statement

Because threads share an address space, there is automatic memory protection between threads.

Answer: False

Explanation: The slides clearly say there is **no automatic memory protection** between threads in the same process—threads are meant to cooperate and can interfere.

---

13) Statement

Race conditions can happen when multiple threads write/update shared data without mutual exclusion.

Answer: True

Explanation: This is a core disadvantage: shared memory can lead to race conditions unless you use mutual exclusion/synchronization.

---

14) Statement

Many library functions are thread-safe because they avoid shared global/static data or use synchronization.

Answer: True

Explanation: The slides note that thread-safe functions avoid shared data or protect it with synchronization.

---

15) Statement

A severe error in one thread (like a segmentation fault) can terminate the whole process.

Answer: True

Explanation: Threads live inside one process; a fatal crash typically kills the process and all its threads.

---

16) Statement

In a multithreaded server, a dispatcher thread can assign incoming requests to worker threads, and workers may block on disk I/O while other threads continue.

Answer: True

Explanation: The slides describe this exact pattern: dispatcher wakes an idle worker; worker may block on read; other threads keep running.

---

17) Statement

User-level threads are created and scheduled by the kernel, so the kernel always knows about each user thread.

Answer: False

Explanation: User-level threads are managed by a **user-space thread library**. The kernel may only see the process (or fewer kernel threads), not each user thread.

---

18) Statement

With user-level threads, if one user thread makes a blocking system call, the entire process may block because the kernel is unaware of the user threads.

Answer: True

Explanation: This is a classic limitation of pure user-level threading described in the slides.

---

19) Statement

In the Many-to-One (M:1) model, many user threads map to one kernel thread, so blocking and lack of multicore parallelism are major limitations.

Answer: True

Explanation: If one user thread blocks, the whole process blocks; and only one kernel thread means no true parallel execution on multicore.

20) Statement

If a thread calls `exec()` in a multithreaded program, only that thread is replaced, while other threads keep running the old program.

Answer: False

Explanation: The slides state that `exec()` replaces the **entire process**, which terminates all existing threads and overwrites process memory (including global variables and pthread objects like mutexes).

If you want, I can also generate **another set of 20** that focuses specifically on **(M:1 vs 1:1 vs M:N)** and **Pthreads operations (create, join, cancel, fork/exec behavior)**—still True/False and still challenging.

Sources



Lec_11_process_synch1.pptx
Presentation

Here are **20 challenging True/False questions** based on your **Process Synchronization -1 (Week 11)** slides Lec_11_process_synch1 .
(Statements avoid negatives/double negatives; false ones are plausible; explanations are in simple English.)

---

1) Statement

Cooperating processes can affect each other because they share data/resources or their execution depends on each other.

Answer: True

Explanation: The slides define cooperating processes as those that can affect/be affected by others, often through shared data/resources or dependent execution.

---

2) Statement

Process synchronization focuses on ensuring orderly execution of cooperating processes so shared data stays consistent.

Answer: True

Explanation: That is the core definition: synchronization mechanisms prevent inconsistent results when multiple processes use shared data.

---

3) Statement

Synchronization is relevant for threads as well as processes because threads share an address space and communicate through shared variables.

Answer: True

Explanation: Threads can directly access shared variables, so they can also cause race conditions unless synchronized.

---

4) Statement

In the producer–consumer example, `ctr` helps decide whether the buffer is full or empty.

Answer: True

Explanation: `ctr` tracks how many items are currently in the buffer, so producer/consumer use it for full/empty checks.

---

5) Statement

`ctr++` is an atomic action in the producer code because it is written as one high-level statement.

Answer: False

Explanation: The slides break `ctr++` into multiple machine-level steps (load, increment, store). Those steps can interleave with another process and produce wrong results.

---

6) Statement

Interleaving of instructions can cause the final value of a shared variable to depend on which process writes last.

Answer: True

Explanation: The producer–consumer and bank balance examples show that different instruction orders can produce different final values.

---

7) Statement

A race condition occurs when multiple cooperating processes/threads update shared data concurrently, and the final value depends on timing.

Answer: True

Explanation: This is the slide definition: outcome depends on which thread/process "wins" the timing race.

---

8) Statement

A critical section is a part of a program where shared data (like a variable, file, or database record) may be accessed or updated.

Answer: True

Explanation: The slides define critical section exactly as the code that accesses/updates shared resources.

---

9) Statement

The critical-section problem exists because multiple processes may try to execute their critical sections at the same time.

Answer: True

Explanation: If they overlap, shared data updates can interleave and become inconsistent, so a protocol is needed.

---

10) Statement

A typical solution structure includes an entry section (request permission), a critical section, an exit section (release control), and a remainder section.

Answer: True

Explanation: This structure is shown explicitly in the slides.

---

11) Statement

Mutual exclusion means that when one process is inside its critical section, no other cooperating process can be inside its critical section.

Answer: True

Explanation: Mutual exclusion is the key safety property that prevents overlapping critical sections.

---

12) Statement

Progress means the choice of who enters the critical section next depends only on processes currently trying to enter, and the decision does not get postponed forever.

Answer: True

Explanation: Progress prevents deadlock-like behavior where everyone waits and no one enters.

---

13) Statement

Bounded waiting ensures that once a process requests to enter its critical section, it will eventually be allowed after a limited number of other entries.

Answer: True

Explanation: This prevents starvation: a process can’t be postponed forever.

---

14) Statement

Strict alternation guarantees mutual exclusion, but it can still force a process to wait even when the other process is in its remainder section.

Answer: True

Explanation: Strict alternation enforces turn-taking. If it's not your turn, you wait—even if the other process is not interested in the critical section right now.

---

15) Statement

The flags algorithm improves over strict alternation by letting a process express its intent to enter the critical section using a boolean flag.

Answer: True

Explanation: Each process sets its flag to true to show interest; the other flag is checked before entering.

---

16) Statement

The basic flags algorithm always guarantees progress because if both processes raise their flags together, one will still enter.

Answer: False

Explanation: If both set their flags true at the same time, both can end up spinning while waiting for the other's flag to become false. That situation can block progress.

---

17) Statement

Peterson's algorithm combines flags and a shared `turn` variable to satisfy mutual exclusion, progress, and bounded waiting for two processes.

Answer: True

Explanation: The slides explicitly list these three properties as provided by Peterson's solution (for two processes).

---

18) Statement

Peterson's algorithm is mainly theoretical because busy-waiting is inefficient and modern CPU memory behaviors can break its assumptions.
Answer: True
Explanation: The slides mention busy-wait inefficiency and that it may not work on modern architectures due to load/store handling.

---

19) Statement

A mutex is an OS-provided synchronization tool that protects a critical section by allowing only one thread/process to hold the lock at a time.
Answer: True
Explanation: Mutex locking enforces one-at-a-time access to the protected code/data.

---

20) Statement

A mutex is safe to destroy when it is unlocked and no threads are blocked waiting to lock it.
Answer: True
Explanation: The slides state safe destruction requires: nobody owns it (unlocked) and nobody is waiting on it.

If you want, I can create **another 20** that focus *only* on:

- strict alternation vs flags vs Peterson (edge cases & properties), and
- mutex usage pitfalls (owner unlock, double lock/unlock, destroy rules),
  still in the same True/False + explanation format.

Sources

Lec_12_process_synch2.pptx
Presentation

Here are **20 challenging True/False questions** from your **Process Synchronization -2 (Week 12)** slides Lec_12_process_synch2 .
(Each has a simple-English explanation; false statements are believable; no negative/double-negative wording.)

---

1) Statement

A semaphore is an OS synchronization tool that can coordinate access for multiple processes more flexibly than a basic mutex lock.
Answer: True
Explanation: The slides present semaphores as OS-provided tools for n-process synchronization and more sophisticated coordination than simple locking.

---

2) Statement

A semaphore is an integer variable that is accessed only through the atomic operations `wait()` **and** `signal()` **.**
Answer: True
Explanation: The key rule is that you don't read/write semaphore values directly; only `wait()` and `signal()` are allowed, and they must be atomic.

---

3) Statement

In the busy-wait semaphore definition, `wait(S)` **repeatedly checks the semaphore value until it becomes positive, then decreases it.**
Answer: True
Explanation: That version spins in a loop (`while S <= 0;`) and then performs `S--` .

---

4) Statement

Atomicity of `wait()` **and** `signal()` **prevents two processes from updating the semaphore value in an interleaved way.**

Answer: True

Explanation: If those operations could interleave, the semaphore value could become incorrect and mutual exclusion could fail.

---

5) Statement

A binary semaphore can take values 0 and 1 and is commonly used to allow one process at a time to access a resource.

Answer: True

Explanation: The slides describe binary semaphores exactly this way and relate them to mutex-style mutual exclusion.

---

6) Statement

A counting semaphore is useful for managing a pool of identical resources by allowing up to a fixed number of concurrent users.

Answer: True

Explanation: Its value represents how many resources are available; multiple processes can proceed as long as the count remains above 0.

---

7) Statement

For a resource pool of size `k`, initializing a counting semaphore to `k` represents that all resources are available at the start.

Answer: True

Explanation: The slides say it's initialized to the number of available resources.

---

8) Statement

Queue-based semaphore waiting replaces busy waiting by blocking a process and placing it in a semaphore's wait queue when the value is not positive.

Answer: True

Explanation: Instead of wasting CPU cycles spinning, the process sleeps (blocks), and the scheduler runs someone else.

---

9) Statement

In the queue-based semaphore implementation, a negative semaphore value represents how many processes are waiting.

Answer: True

Explanation: The slides state that a negative `S.value` indicates the number of processes waiting on that semaphore.

---

10) Statement

Busy-wait semaphores can be a reasonable choice when critical sections are very short and expected wait time is small.

Answer: True

Explanation: The slides compare the two approaches and note busy-wait can be better for short waits because it avoids context-switch overhead.

---

11) Statement

Queue-based waiting is usually better for long critical sections because it saves CPU cycles during long waits.

Answer: True

Explanation: If waiting is long, blocking is more efficient than spinning.

---

12) Statement

To force statement B in `Pj` to run after statement A in `Pi`, you can initialize a semaphore to 0, signal it after A, and wait on it before B.

Answer: True

Explanation: This is the standard “ordering” pattern shown in the slides.

---

13) Statement

Semaphores can express a chain of ordering constraints across multiple processes by using more than one semaphore.

Answer: True

Explanation: The slides give an example where `P1` waits on one semaphore and `P2` waits on another to enforce a sequence across `P3 → P2 → P1`.

---

14) Statement

Deadlock happens when a set of processes are waiting for events that only other processes in the same set can cause.

Answer: True

Explanation: That's the slide definition: everyone waits, and only someone else in the waiting group could release them.

---

15) Statement

Starvation is indefinite blocking where a process keeps waiting because resources keep becoming available to others first.

Answer: True

Explanation: Starvation is about unfairness: a process can wait forever even though the system continues running.

---

16) Statement

A semaphore solution becomes easy to understand when `wait()` and `signal()` calls are scattered throughout many processes.

Answer: False

Explanation: The slides say the opposite: scattered semaphore operations can be hard to reason about, making errors more likely.

---

17) Statement

Two processes can deadlock if each holds one semaphore and waits for the other semaphore held by the other process.

Answer: True

Explanation: The slides show the classic pattern: `P0` gets `S` then waits on `Q`, while `P1` gets `Q` then waits on `S`.

---

18) Statement

Mutual exclusion can break if a process swaps the order of semaphore operations and signals before waiting.

Answer: True

Explanation: The slide example shows that reordering `wait()` / `signal()` can allow two processes to be in the critical section at the same time.

---

19) Statement

POSIX provides unnamed semaphores (shared among threads in one process) and named semaphores (shareable across unrelated processes).

Answer: True

Explanation: The slides distinguish these two semaphore types and explain their scope.

---

20) Statement

In the bounded-buffer semaphore solution, `empty` tracks empty slots, `full` tracks filled slots, and `mutex` enforces mutual exclusion while updating the buffer indices.

Answer: True

Explanation: `empty/full` provide correct coordination between producer and consumer, while `mutex` prevents race conditions in the shared buffer updates (`in`, `out`, and buffer array).

If you'd like, I can generate **another 20** that focus only on the **classic problems** (bounded buffer, readers–writers, dining philosophers) with more "edge-case" style statements about

deadlock and starvation—still True/False + simple explanations.

Sources

Lec_13_deadlocks.pptx
Presentation

Here are **20 challenging True/False questions** based on your uploaded **Week 13: Deadlocks** slides Lec_13_deadlocks .
(Each has a simple-English explanation; false statements are plausible; no negative/double-negative wording.)

---

1) Statement

A deadlock occurs when each process in a set holds at least one resource and waits for a resource held by another process in the same set.
Answer: True
Explanation: This is the core definition given: the processes form a closed waiting chain where nobody can proceed.

---

2) Statement

Deadlocks commonly appear in concurrent systems because multiple processes/threads can create cycles of dependencies on resources.
Answer: True
Explanation: Concurrency creates interleavings and "who holds what" situations that can form cycles.

---

3) Statement

Deadlocks typically involve non-preemptable resources, where taking a resource away can cause problems.

Answer: True

Explanation: If the OS cannot safely take a resource back (like a printer mid-print), deadlock becomes more likely.

---

4) Statement

A preemptable resource can be taken away from a process and reassigned without harming correctness.

Answer: True

Explanation: The slides give examples like CPU and memory as resources that can often be preempted safely.

---

5) Statement

The four necessary conditions for deadlock are mutual exclusion, hold-and-wait, non-preemption, and circular wait.

Answer: True

Explanation: The slides list these as the four conditions that must occur together for deadlock to arise.

---

6) Statement

Mutual exclusion means a resource can be used by only one process at a time.

Answer: True

Explanation: If resources were freely shareable, processes would not get stuck waiting for exclusive access.

---

7) Statement

Hold-and-wait means a process holds resources while requesting additional resources.

Answer: True

Explanation: This “hold while requesting more” behavior builds waiting chains.

---

8) Statement

Circular wait means processes form a cycle where each process waits for a resource held by the next process in the cycle.

Answer: True

Explanation: A circular chain is what makes the waiting impossible to resolve without intervention.

---

9) Statement

Deadlock prevention aims to ensure that at least one of the four necessary conditions never holds.

Answer: True

Explanation: Prevention is a design-time strategy: break one condition so deadlock cannot form.

---

10) Statement

Denying mutual exclusion is easy to apply to all resources, including printers and mutex locks.

Answer: False

Explanation: Some resources are inherently non-shareable (printer, mutex). Making them shareable is often impossible or unsafe.

---

11) Statement

One hold-and-wait prevention approach allocates all required resources to a process before it starts executing.

Answer: True

Explanation: This ensures the process never holds some resources while waiting for others, but it can waste resources.

12) Statement

Requiring a process to release all currently held resources before requesting new ones can reduce deadlock risk but can cause starvation.

Answer: True

Explanation: This breaks hold-and-wait, but a process may repeatedly release and fail to reacquire resources in time.

---

13) Statement

Preemption-based prevention can work well for resources whose state can be saved and restored easily.

Answer: True

Explanation: The slides explain this fits preemptable resources (like CPU state) better than devices like printers/tape drives.

---

14) Statement

Ordering resource requests by assigning each resource type a unique number and forcing requests in increasing order can prevent circular wait.

Answer: True

Explanation: If everyone requests resources in the same global order, cycles cannot form.

---

15) Statement

Resource ordering is a sufficient method to avoid deadlock, but some systems can still run safely without it.

Answer: True

Explanation: The slides say ordering is sufficient but not necessary—deadlock doesn't always happen even without ordering.

---

16) Statement

In a Resource Allocation Graph (RAG), processes are rectangles and resources are circles.
Answer: False
Explanation: The slides define **processes as circles** and **resources as rectangles**. Mixing these up changes the meaning of edges.

---

17) Statement

In a RAG, a request edge goes from a process to a resource, and an assignment edge goes from a resource to a process.
Answer: True
Explanation: A process "asks" a resource (request), and a resource "is allocated to" a process (assignment).

---

18) Statement

If a RAG has no cycles, the system is free of deadlock.
Answer: True
Explanation: The slides state: no cycle ⇒ no deadlock.

---

19) Statement

If a RAG has a cycle and each resource type has only one instance, the cycle implies deadlock.
Answer: True
Explanation: With single instances, a cycle directly means each process is waiting on the next with no alternate instance to break it.

---

20) Statement

If a RAG has a cycle and some resource types have multiple instances, the cycle always implies deadlock.
Answer: False

Explanation: With multiple instances, a cycle indicates **possibility** of deadlock, but the system might still proceed if some process can release a resource and break the cycle.

If you want, I can generate **another 20** that focus specifically on **avoidance (safe state/safe sequence, RAG claim edges, Banker's Algorithm steps + matrices)** and **detection & recovery (wait-for graph, victim selection, rollback, preemption)** from the same Week 13 slides Lec_13_deadlocks .

Sources

Lec_14_memory_mgmt1.p...
Presentation

Here are **20 challenging True/False questions** based on your uploaded **Week 14: Memory Management 1** slides Lec_14_memory_mgmt1 .
(Concept-focused, no negative/double-negative wording, and every False option is plausible.)

---

1) Statement

Memory management aims to ensure fair, secure, orderly, and efficient use of memory.
Answer: True
Explanation: This is the stated purpose in the slides. The OS tries to share memory fairly, prevent unsafe access, and use memory efficiently.

---

2) Statement

A running program generates a stream of memory addresses, and every memory reference passes through the MMU.

Answer: True

Explanation: The CPU produces addresses while executing instructions, and the MMU performs address translation/protection checks.

---

3) Statement

CPU registers are extremely fast and very small compared to main memory.

Answer: True

Explanation: The memory hierarchy shows registers are fastest and smallest, while main memory is larger but slower.

---

4) Statement

The base register defines the starting physical address of a process's memory region, and the limit register defines its address range size.

Answer: True

Explanation: Base gives "where the process begins in RAM," and limit gives "how big the allowed region is."

---

5) Statement

With base and limit registers, the CPU checks each user-mode memory access to ensure it lies within the allowed range.

Answer: True

Explanation: This is hardware-based protection to stop a process from touching OS memory or another process's memory.

---

6) Statement

Symbolic addresses appear in source code, relocatable addresses appear in object code, and absolute addresses appear after linking/loading.

Answer: True

Explanation: The slides describe this program-lifetime progression of addresses.

7) Statement

Compile-time binding produces relocatable code so the program can be placed anywhere without recompilation.

Answer: False

Explanation: Compile-time binding generates **absolute** code if the final location is known; if that location changes, recompilation is needed. Relocatable code is linked to load-time binding.

---

8) Statement

Execution-time binding requires hardware support for run-time address mapping, such as a relocation register.

Answer: True

Explanation: If the process can move during execution, the system must translate addresses dynamically using hardware support.

---

9) Statement

Dynamic linking reduces disk and memory use by keeping libraries separate and linking them at load time or when first needed.

Answer: True

Explanation: The slides list these advantages: smaller binaries, shared library usage, and less memory.

---

10) Statement

Static linking usually loads programs faster because the executable already contains the library code.

Answer: True

Explanation: Since required library code is included, less linking work is needed at load/runtime, so startup can be simpler.

---

11) Statement

A logical address is generated by the CPU, and a physical address refers to an actual location in RAM.

Answer: True

Explanation: The slides define logical vs physical and say the MMU maps logical → physical.

---

12) Statement

The MMU generalizes the base register by adding the relocation register value to every logical address before accessing memory.

Answer: True

Explanation: That's the run-time mapping mechanism shown: physical = relocation + logical.

---

13) Statement

Swapping allows total memory needs of processes to exceed available RAM by moving process images to a backing store and later bringing them back.

Answer: True

Explanation: This is exactly the purpose of swapping described in the slides.

---

14) Statement

In swapping, transfer time is the major cost and is roughly proportional to the amount of memory moved.

Answer: True

Explanation: The slides emphasize that moving more MB means more time, because disk transfer dominates.

---

15) Statement

Contiguous memory allocation loads each process into one continuous block of RAM and uses base and limit registers for mapping.

Answer: True

Explanation: This is the definition of contiguous allocation, and base/limit are the standard hardware support.

---

16) Statement

MFT divides memory into fixed partitions, and each partition can contain exactly one process at a time.

Answer: True

Explanation: Fixed partitioning assigns one partition per process.

---

17) Statement

MFT commonly causes internal fragmentation because a small process still occupies its full fixed partition.

Answer: True

Explanation: Internal fragmentation is wasted space inside an allocated partition when the process is smaller than the partition.

---

18) Statement

MVT creates partitions dynamically to match process sizes, which avoids internal fragmentation but can lead to external fragmentation.

Answer: True

Explanation: Variable partitions fit processes closely, but leave scattered holes between partitions over time.

---

19) Statement

Compaction reduces external fragmentation by shifting processes to merge scattered free spaces into one large free block, and it requires run-time address binding.

Answer: True

Explanation: Moving processes changes their physical location, so the system needs dynamic relocation to keep addresses valid.

---

20) Statement

Paging avoids external fragmentation by splitting logical memory into pages and physical memory into frames of the same size, and mapping pages to any free frames.

Answer: True

Explanation: Because pages can go into non-contiguous frames, you don't need one big contiguous free block, so external fragmentation is avoided (though small internal waste within the last page can still happen).

If you want, I can generate **another 20** that are more "calculation-style" from the slide problems (page number/offset, number of pages/frames, address format bits) while still True/False and concept-heavy.

Sources

Lec_15_memory_mgmt2.p...
Presentation

Here are **20 challenging True/False questions** based on your uploaded **Week 15: Memory Management 2 (Virtual Memory, Demand Paging, Page Faults, Page Replacement)** slides

Lec_15_memory_mgmt2 .

(Concept-focused, no negative/double-negative wording, and the False ones are believable.)

---

1) Statement

Virtual memory allows a process to run even when its entire address space is not in main memory.

Answer: True

Explanation: Virtual memory is designed so only needed parts (pages) are kept in RAM, while the rest can stay on disk.

---

2) Statement

Demand paging loads a page into memory only when the process actually references it.

Answer: True

Explanation: Demand paging starts with pages not loaded and brings them in when a page fault happens.

---

3) Statement

In demand paging, the valid/invalid (present) bit in a page table entry indicates whether a page is currently resident in memory.

Answer: True

Explanation: If the bit indicates "present/valid," the page is in RAM; otherwise, the page is still in backing store and causes a page fault when accessed.

---

4) Statement

A page fault occurs when the CPU references a page that is currently located in the backing store instead of RAM.

Answer: True

Explanation: The slide definition says page fault happens when the needed page is not in memory.

---

5) Statement

When a page fault happens, the OS can either abort the process or bring the required page into memory depending on whether the reference is valid.

Answer: True

Explanation: If the reference is invalid (illegal address), the process is aborted; if valid but page not present, the OS loads it.

---

6) Statement

Servicing a page fault typically includes locating the page on disk, loading it into a free frame, updating the page table, and restarting the faulting instruction.

Answer: True

Explanation: These steps are explicitly listed in the slides as the page-fault service routine.

---

7) Statement

When no free frame exists during a page fault, the OS performs page replacement by selecting a victim frame to evict.

Answer: True

Explanation: If RAM is full, the OS must free space by removing some page and bringing the needed one in.

---

8) Statement

Pre-paging brings pages into memory before they are referenced, aiming to reduce the number of page faults.

Answer: True

Explanation: Pre-paging tries to guess what will be needed soon (for example, bringing the next page too).

---

9) Statement

Request paging relies on the user/programmer to specify which pages should be loaded, but users do not always predict well.

Answer: True

Explanation: The slides mention this approach and its drawback: the user may not know what is best.

---

10) Statement

Fetch strategy decides when to bring a page into memory, placement strategy decides where to place it in memory, and replacement strategy decides what to evict.
Answer: True
Explanation: The slides group paging decisions into fetch, placement, and replacement.

---

11) Statement

The principle of optimality says the best page to evict is the one that will be used farthest in the future.
Answer: True
Explanation: That replacement choice minimizes future page faults in theory.

---

12) Statement

The optimal replacement algorithm is practical in real systems because the OS can know future memory references accurately.
Answer: False
Explanation: The OS generally cannot see the future reference string, so optimal is used mainly as a benchmark for comparison.

---

13) Statement

FIFO page replacement evicts the page that has been in memory the longest.
Answer: True
Explanation: FIFO uses arrival time (oldest in memory leaves first), typically implemented with a queue.

---

14) Statement

FIFO can suffer from Belady's anomaly where increasing the number of frames can increase the number of page faults for some reference strings.

Answer: True

Explanation: The slides show an example where 4 frames causes more page faults than 3 frames under FIFO.

---

15) Statement

Belady's anomaly can occur in FIFO, but optimal replacement does not show this behavior.

Answer: True

Explanation: Optimal always makes the best future choice, so giving it more frames cannot make it worse.

---

16) Statement

LRU replaces the page that has not been used for the longest time in the past, using recent history as an approximation of future behavior.

Answer: True

Explanation: LRU assumes locality: pages used recently are likely to be used again soon.

---

17) Statement

LRU always produces the same page fault rate as the optimal algorithm on any reference string.

Answer: False

Explanation: LRU is an approximation. It often performs well, but it can still make different choices than optimal and can fault more.

---

18) Statement

A frequently used page can still get evicted in FIFO because FIFO ignores how recently or how often a page is referenced.
Answer: True
Explanation: FIFO only cares about age in memory, so even a popular page can be removed if it is the oldest.

---

19) Statement

Local replacement chooses a victim frame from the same process that triggered the page fault, while global replacement can evict a frame belonging to another process.
Answer: True
Explanation: The slides define these two categories of replacement scope.

---

20) Statement

A dirty bit in a page table entry helps reduce disk writes by allowing clean (unchanged) pages to be discarded without writing them back.
Answer: True
Explanation: If the page was never modified, the OS can drop it and reload from disk later, avoiding an unnecessary write-back.

If you want, I can generate **another 20** that are more "problem-style" using the slide reference strings (FIFO vs Optimal vs LRU outcomes, Belady anomaly interpretation, and dirty-bit scenarios), still in True/False format.

Sources



Lec_16_Segmentation.pptx
Presentation

Here are **20 challenging True/False questions** based on your uploaded **Week 16: Segmentation** slides Lec_16_Segmentation .
(Concept-focused, no negative/double-negative statements, and the false ones are plausible.)

---

1) Statement

Segmentation is a memory-management scheme that matches the programmer's view of memory by dividing a program into logical units.
Answer: True
Explanation: In segmentation, a program is seen as a collection of meaningful parts like procedures, stack, globals, etc., rather than one flat array of bytes. Lec_16_Segmentation

---

2) Statement

A segment is a logical unit of a program such as a function, stack, symbol table, local variables, or global variables.
Answer: True
Explanation: The slides list these as examples of segments. Lec_16_Segmentation

---

3) Statement

In segmentation, each segment is loaded into memory as one contiguous block, while different segments can be placed in different locations.
Answer: True
Explanation: Each segment must be contiguous internally, but segments do not have to be adjacent to each other in physical memory. Lec_16_Segmentation

---

4) Statement

Segmentation loads all segments of a process into one single contiguous physical region.

Answer: False

Explanation: Each segment is contiguous, but segments can be scattered across memory in different partitions. Lec_16_Segmentation

---

5) Statement

In segmentation, partitions used to load segments are created dynamically based on the segment sizes.

Answer: True

Explanation: The slides state partitions are created dynamically, unlike fixed partitioning.

Lec_16_Segmentation

---

6) Statement

Segments can have different lengths, so the system keeps a maximum segment length limit.

Answer: True

Explanation: The slides mention segments can have different lengths but there is some max segment length. Lec_16_Segmentation

---

7) Statement

A logical address in segmentation has the form ⟨segment-number, offset⟩.

Answer: True

Explanation: Segment number selects the segment, and offset selects a location inside it.

Lec_16_Segmentation

---

8) Statement

The segment number in a logical address is used as an index into the segment table.

Answer: True

Explanation: The segment table stores mapping information for each segment, and the segment number chooses the table entry. Lec_16_Segmentation

9) Statement

A segment table entry contains a base (starting physical address) and a limit (segment length).

Answer: True

Explanation: Base tells where the segment begins in physical memory; limit is used to check whether an offset is legal. Lec_16_Segmentation

---

10) Statement

The physical address for a segmented reference is computed by adding the segment base to the offset.

Answer: True

Explanation: The base is the physical starting address; adding the offset gives the exact physical memory location. Lec_16_Segmentation

---

11) Statement

The limit field in the segment table is used to detect addressing errors when the offset exceeds the segment size.

Answer: True

Explanation: If offset goes beyond the segment length, it's an invalid memory reference.

Lec_16_Segmentation

---

12) Statement

Because segments are variable-sized and loaded into dynamic partitions, segmentation commonly leads to external fragmentation.

Answer: True

Explanation: Free space can become scattered into small holes between segments.

Lec_16_Segmentation

---

13) Statement

Compaction can reduce external fragmentation in segmentation by shifting segments to combine scattered free space into one block.

Answer: True

Explanation: The slides describe compaction as shuffling segments to gather free spaces together. Lec_16_Segmentation

---

14) Statement

Compaction in segmentation works best when the system uses run-time address binding so segments can be relocated during execution.

Answer: True

Explanation: If segments move, addresses must still translate correctly, so dynamic relocation/run-time binding is needed. Lec_16_Segmentation

---

15) Statement

Paged segmentation reduces external fragmentation by dividing each segment into fixed-size pages and mapping those pages to frames.

Answer: True

Explanation: Paging segments makes allocation frame-based, so you don't need variable-size holes to fit segments. Lec_16_Segmentation

---

16) Statement

In paged segmentation, each process has one segment table and exactly one page table for the whole process.

Answer: False

Explanation: The slides state: one **segment table** plus **multiple page tables** (one page table per segment). Lec_16_Segmentation

---

17) Statement

In paged segmentation, the offset inside a segment is split into a page number `p` and a page offset `d'`.

Answer: True

Explanation: The segment offset `d` is divided so the system can index the segment's page table and then locate the byte within the page. Lec_16_Segmentation

---

18) Statement

In paged segmentation, the segment number selects a segment-table entry that points to the page table for that segment.

Answer: True

Explanation: After selecting the segment, the system uses that segment's page table to map the page number to a frame. Lec_16_Segmentation

---

19) Statement

In paged segmentation, the physical address is formed as ⟨frame-number, page-offset⟩ after the page number is translated through the segment's page table.

Answer: True

Explanation: The page table gives the frame `f`, and the final address uses that frame plus the within-page offset `d'`. Lec_16_Segmentation

---

20) Statement

For a logical address (s, d), legality is checked by ensuring `d` is within the segment length before translating it into (p, d').

Answer: True

Explanation: The segment length (limit) defines valid offsets for that segment; only then is the offset interpreted as page number + page offset in paged segmentation.

Lec_16_Segmentation

Sources

